# Minor Assignment — 01

Game Programming with C++ (CSE 3545)

*Department of Computer Science and Engineering, Institute of Technical Education & Research, SOA Deemed to be University*

*All outputs below were verified by compiling each snippet with g++ 13.3.0 on Linux.*

## Q1. swap() with reference parameters

```
void swap (int &a, int &b) {
    int temp; temp = a;
    a = b;
    b = temp;
}
int main () {
    int i = 0, j = 1;
    swap (i, j);
    cout << i << " " << j << endl;
}
```

### Output

```
1 0
```

**Explanation.** The parameters `int &a, int &b` are *references*, which means that inside `swap` the names a and b refer to the *same memory locations* as `i` and `j` in `main`. So assigning to a/b actually changes `i`/`j`. After the swap: `i = 1, j = 0`.

> ***Note:*** *If the parameters had been declared* `int a, int b` *(pass-by-value), each would be a local copy and the swap would have no visible effect — the output would have been* `0 1`*.*

## Q2. Mixing pass-by-value and pass-by-reference for the same argument

```
void myfun(int i, int &k) {
    i = 1;
    k = 2;
}
int main () {
    int x = 0;
    myfun (x, x);
    cout << x << endl;
    return 0;
}
```

### Output

```
2
```

**Explanation.** The call `myfun(x, x)` passes the same variable `x` to both parameters, but through different mechanisms:

```
 - i  -> pass by VALUE:      'i' is a fresh copy; 'i = 1;' touches only the copy,
                              x stays 0.
 - k  -> pass by REFERENCE:  'k' is an alias for x itself; 'k = 2;' overwrites x.

Execution order:   i = 1    (no effect on x)
                   k = 2    (x becomes 2)

Final value of x  =>  2
```

## Q3. Scope resolution operator with nested blocks

```
int x = 1;              // global x
```

```
void fun(){
    int x = 2;          // 1st local x shadows the global
    {
        int x = 3;      // 2nd local x shadows the outer local
        cout << ::x << endl;
    }
}
int main(){ fun(); return 0; }
```

**Output**

```
1
```

**Explanation.** The unary operator `::` is the *scope-resolution operator*. `::x` explicitly selects the identifier `x` from the **global** namespace, bypassing *every* local scope, no matter how deeply nested. Hence even with two local `x`'s (values 2 and 3) in sight, `::x` refers to the global `x = 1`.

## Q4. Global (static-storage) array default initialisation

```
int x[100];              // global array
int main()
{
    cout << x[99] << endl;
}
```

**Output**

```
0
```

**Explanation.** `int x[100]` is declared at **file (global) scope**, so it has *static storage duration*. The C++ standard guarantees that such objects are **zero-initialised** before `main()` starts. Every element — including `x[99]` — is therefore 0.

> ***Note:*** *If the same array had been declared inside* `main()` *as a local (automatic) variable, its contents would be* ***indeterminate*** *and* `x[99]` *would likely print a garbage value.*

## Q5. Cube via pass-by-reference on a double

```
void Cube(double &y) {
    y = y*y*y;
}
int main()
{
    double g = 4.0;
    Cube(g);
    cout << g << endl;
    return 0;
}
```

**Output**

```
64
```

**Explanation.** `y` is a reference, so `y = y*y*y` writes back into `g` itself: `g` becomes 4.0 × 4.0 × 4.0 = 64.0. `cout`'s default formatting for a `double` with no fractional part prints it as `64`, not `64.0` — the trailing zeros are suppressed unless `fixed` or `setprecision` is used.

## Q6. Constructor with a default argument used as a default constructor

```
class Sample {
public:
    Sample(int x = 10) {
        cout << "Value: " << x << endl;
    }
};
```

```
int main() {
    Sample obj;
    return 0;
}
```

**Output**

```
Value: 10
```

**Explanation.** `Sample obj;` needs a constructor callable with **no arguments**. Because every parameter of `Sample(int x = 10)` has a default value, the constructor can indeed be invoked without any argument, and in that case `x` takes its default value 10. The constructor body prints `Value: 10`.

> ***Note:*** *A constructor that can be called with no arguments is (by definition) the class's* ***default constructor****, whether it has 0 parameters or N parameters all with defaults. You may not have both forms at once — that creates an ambiguity.*

## Q7. Static local object lifetime

```
class A {
public:
    A()  { cout << "A "; }
    ~A() { cout << "~A "; }
};
void func() {
    static A obj;       // constructed ONCE, on first call
}
int main() {
    func();
    func();
    cout << "Main ";
    return 0;
}
```

**Output**

```
A Main ~A
```

**Explanation.** A function-local `static` object is constructed the **first time** control reaches its declaration and is destroyed when the *program* terminates, not when the function returns.

```
func();   // first call  -> constructor runs       prints:  "A "
func();   // second call -> obj already alive, skipped
cout << "Main ";                                  prints:  "Main "
return 0; // program ending -> static obj destroyed prints:  "~A "
```

> ***Note:*** *That's why the destructor output* `~A` *appears* ***after*** `Main` *— destruction happens during program teardown, after* `return 0`*.*

## Q8. Explicitly calling a destructor on an automatic object

```
class MyClass{
public:
    ~MyClass() {
        cout << "My destructor" << endl;
    }
};
void main()         // NOTE: non-standard signature
{
    MyClass obj;
    obj.~MyClass(); // explicit destructor call
}
```

**Output (two lines)**

```
My destructor
My destructor
```

**Explanation.** Two separate things happen:

```
1. obj.~MyClass();                  // EXPLICIT destructor call
     -> body runs, prints "My destructor" once

2. end of main()  -> obj goes out of scope
     -> the compiler AUTOMATICALLY calls the destructor a second time
     -> prints "My destructor" again
```

> ***Note:*** *Two subtle issues for the graders:*
> • `void main()` *is* ***not legal ISO C++****; the standard says* `int main()`*. Strict compilers (*`g++ -pedantic`*) reject it; MSVC tolerates it as an extension.*
> • *Calling the destructor manually on an automatic object leaves it in a* ***destroyed state****, but the compiler will still destroy it again at scope end — this is formally* ***undefined behaviour****. The visible output on common compilers is the destructor message printed twice.*

## Q9. malloc() does NOT call the C++ constructor

```
class Test {
public:
    Test() { cout << "Constructor called"; }
};
int main()
{
    Test *t = (Test *) malloc(sizeof(Test));
    return 0;
}
```

### Output

```
(no output - the program prints nothing)
```

**Explanation.** `malloc()` is a C library function. It only **allocates raw, uninitialised memory** — it does **not** construct a C++ object inside that memory. The class's constructor is therefore never invoked and the `"Constructor called"` string is never printed.

> ***Note:*** *This is precisely why C++ added the* `new` *operator:* `new` *combines allocation and construction in one step (see Q10).*

## Q10. new invokes the constructor

```
class Test {
public:
    Test() { cout << "Constructor called"; }
};
int main()
{
    Test *t = new Test();
    return 0;
}
```

### Output

```
Constructor called
```

**Explanation.** The expression `new Test()` performs **two** actions: (1) it allocates enough memory on the free store to hold a `Test` object, and (2) it calls `Test::Test()` on that memory. The constructor body therefore runs and prints `Constructor called`.

> ***Note:*** *Comparing Q9 and Q10 is a classic exam question:* `malloc + free` *(C) vs.* `new + delete` *(C++). Only the latter pair cooperates with constructors and destructors.*

## Q11. Employee class — Annual Income-Tax Calculator

The program below defines an `Employee` class with:

• A **default constructor** that initialises every field to safe placeholder values.
• A **parameterised constructor** whose parameters all carry **default arguments** — so it can be called with just a name or with all six details.
• `calculateTax()` applying the FY 2024-25 New-Regime slabs, 87A rebate, surcharge (10% / 15%) and 4% Health & Education cess.
• `printTax()` emitting a tabular report in the format shown in the question's sample image.

### Slab rules applied (New Regime, FY 2024-25)

```
  Income <= 300000            : NIL
  300000 < Income < 700000    : 5% of (Income - 300000)
  700000 <= Income < 1000000  : 20000  + 10% of (Income - 700000)
  1000000 <= Income < 1200000 : 50000  + 15% of (Income - 1000000)
  1200000 <= Income < 1500000 : 80000  + 20% of (Income - 1200000)
  Income >= 1500000           : 140000 + 30% of (Income - 1500000)

Rebate u/s 87A:  full tax waived if taxable income <= 7 L
Surcharge:       +10% if taxable income >  50 L
                 +15% if taxable income >   1 Cr
Health & Edu Cess: 4% on (tax + surcharge)
Std. Deduction u/s 16(ia): Rs 75000 (New Regime)
```

### Program

```
/* ============================================================
 * Q11 - Payroll Management: Employee Income-Tax Calculator
 * Game Programming with C++ (CSE 3545) - Minor Assignment-01
 * FY 2024-25 (A.Y. 2025-26)  -  New Tax Regime slabs
 * ============================================================ */
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Employee {
private:
    // ---- details ----
    string name, designation, address, pan;
    int    age;
    double grossSalary;

    // ---- computed values ----
    double stdDeduction;
    double taxableIncome;
    double slabTax;           // tax straight from the slab formula
    double rebate;            // u/s 87A
    double taxAfterRebate;
    double surcharge10, surcharge15;
    double cess;              // Health & Education cess @ 4%
    double totalTax;

    // ---- which slab did the taxable income fall into? ----
    int activeSlab;           // 0..5  -> 0 means NIL slab

public:
    /* --------------- Default constructor --------------- */
    Employee() {
        name        = "Not Provided";
        designation = "N/A";
```

```
    address     = "N/A";
    pan         = "N/A";
    age         = 0;
    grossSalary = 0.0;
    resetComputed();
}

/* --- Parameterised constructor with default arguments --- */
Employee(string n,
         string d  = "Asst. Prof.",
         string ad = "N/A",
         string p  = "N/A",
         int    ag = 30,
         double gs = 0.0) {
    name        = n;
    designation = d;
    address     = ad;
    pan         = p;
    age         = ag;
    grossSalary = gs;
    resetComputed();
}

void resetComputed() {
    stdDeduction = taxableIncome = slabTax = rebate = 0.0;
    taxAfterRebate = surcharge10 = surcharge15 = cess = totalTax = 0.0;
    activeSlab = 0;
}

/* ================= Tax computation ================= */
void calculateTax() {
    stdDeduction  = 75000.00;                   // u/s 16(ia) - New Regime
    taxableIncome = grossSalary - stdDeduction;

    // --- slab based tax (FY 2024-25 New Regime) ---
    if (taxableIncome <= 300000) {
        slabTax = 0.0;                          activeSlab = 0;
    } else if (taxableIncome < 700000) {
        slabTax = 0.05 * (taxableIncome - 300000);
        activeSlab = 1;
    } else if (taxableIncome < 1000000) {
        slabTax = 20000 + 0.10 * (taxableIncome - 700000);
        activeSlab = 2;
    } else if (taxableIncome < 1200000) {
        slabTax = 50000 + 0.15 * (taxableIncome - 1000000);
        activeSlab = 3;
    } else if (taxableIncome < 1500000) {
        slabTax = 80000 + 0.20 * (taxableIncome - 1200000);
        activeSlab = 4;
    } else {
        slabTax = 140000 + 0.30 * (taxableIncome - 1500000);
        activeSlab = 5;
    }

    // --- Rebate u/s 87A: taxable income <= 7 L ---
    rebate         = (taxableIncome <= 700000) ? slabTax : 0.0;
    taxAfterRebate = slabTax - rebate;

    // --- Surcharge ---
    surcharge10 = surcharge15 = 0.0;
    if (taxableIncome > 10000000)                   // > 1 Crore
```

```
        surcharge15 = 0.15 * taxAfterRebate;
    else if (taxableIncome > 5000000)               // > 50 Lakhs
        surcharge10 = 0.10 * taxAfterRebate;

    // --- Health & Education cess @ 4% ---
    cess = 0.04 * (taxAfterRebate + surcharge10 + surcharge15);

    // --- Final tax payable ---
    totalTax = taxAfterRebate + surcharge10 + surcharge15 + cess;
}

/* ================= Pretty-printed report ================= */
void printTax() {
    cout << fixed << setprecision(2);
    const int W = 90;                           // total row width
    string line(W, '-');

    // ---- personal block ----
    cout << "Name in Full : "   << name        << '\n'
         << "Designation: "     << designation << '\n'
         << "Office address : " << address     << '\n'
         << "AGE: "             << age         << '\n'
         << "PAN: "             << pan         << '\n';

    cout << line << '\n';
    cout << "|" << setw(W - 2) << right
         << "For the Financial Year 2024-25 (A.Y. 2025-26)" << " |\n";
    cout << line << '\n';

    // ---- salary block ----
    printRow("Gross Salary (Pay +GP+DA+HRA+ALLOWANCES)",
             "Rs.",  grossSalary,   true);
    printRow("Less: Standard Deduction u/s 16(ia)",
             "-Rs.", stdDeduction,  true);
    printRow("Total Income / Taxable income",
             "Rs.",  taxableIncome, true);

    cout << line << '\n';
    cout << "|Calculation of Income Tax" << setw(W - 27) << "|\n";

    // ---- per-slab rows (all shown, only the active one has a value) ----
    printSlab("  Income <= 300000.00 : NIL",
              activeSlab == 0, 0.00);
    printSlab("  300000.00 <= Income < 700000.00   : 5% of (Income - 300000)",
              activeSlab == 1, slabTax);
    printSlab("  700000.00 <= Income < 1000000.00  : 20000 + 10% of (Income - 700000)",
              activeSlab == 2, slabTax);
    printSlab("  1000000.00 <= Income < 1200000.00 : 50000 + 15% of (Income - 1000000)",
              activeSlab == 3, slabTax);
    printSlab("  1200000.00 <= Income < 1500000.00 : 80000 + 20% of (Income - 1200000)",
              activeSlab == 4, slabTax);
    printSlab("  Income > 1500000.00               : 140000 + 30% of (Income - 1500000)",
              activeSlab == 5, slabTax);

    cout << line << '\n';
    printRow("Rebate Total Income less than Rs. 7 Lakhs.",
             "-Rs.", rebate, true);
    cout << line << '\n';

    // ---- tax + add-ons block ----
    printRow("Total Tax",
```

```
                "Rs.", taxAfterRebate, true);
        printRow("Education & Health Cess @ 4% on Income Tax",
                "Rs.", cess, true);
        printSlab2("Add: Surcharge @ 10% (if income => 50 Lakhs)",
                  surcharge10 > 0, surcharge10);
        printSlab2("Add: Surcharge @ 15% (if income is more than => 1 Crore)",
                  surcharge15 > 0, surcharge15);

        cout << line << '\n';
        printRow("Total tax payable", "Rs.", totalTax, true);
        cout << line << '\n';
    }

private:
    // label on the left, "Rs. 12345.67" right aligned at width W
    void printRow(const string& label, const string& prefix,
                  double value, bool /*active*/) {
        const int W = 90;
        cout << "|" << label;
        int used = 1 + (int)label.size();
        int pad  = W - used - 2;                       // leave room for "|"

        // right portion: "<prefix> <value>"  with a fixed layout
        // width of the numeric column ~= 18 chars
        cout << setw(pad - 18) << "" << setw(6) << right << prefix
             << setw(12) << value << "|\n";
    }

    // slab row: either shows "Rs. <val>" or "NA"
    void printSlab(const string& label, bool active, double value) {
        const int W = 90;
        cout << "|" << label;
        int used = 1 + (int)label.size();
        int pad  = W - used - 2;
        if (active)
            cout << setw(pad - 18) << "" << setw(6) << "Rs."
                 << setw(12) << value << "|\n";
        else
            cout << setw(pad) << "" << setw(2) << "NA" << "|\n";
    }

    // surcharge-style row: "Rs. <val>" or "NA"
    void printSlab2(const string& label, bool active, double value) {
        printSlab(label, active, value);
    }
};

/* ================ HR driver ================ */
int main() {
    // Object created via parameterised constructor with default arguments
    Employee emp("Satya Brata Rout",
                 "Asst. Prof.",
                 "N/A",
                 "XA0845QA",
                 36,
                 1275000.00);

    emp.calculateTax();
    emp.printTax();

    return 0;
```

```
}
```

## Program Output (for Satya Brata Rout, gross = Rs 12,75,000)

```
Name in Full : Satya Brata Rout
Designation: Asst. Prof.
Office address : N/A
AGE: 36
PAN: XA0845QA
-----------------------------------------------------------------------------------------
|                                        For the Financial Year 2024-25 (A.Y. 2025-26) |
-----------------------------------------------------------------------------------------
|Gross Salary (Pay +GP+DA+HRA+ALLOWANCES)                                 Rs.  1275000.00|
|Less: Standard Deduction u/s 16(ia)                                     -Rs.    75000.00|
|Total Income / Taxable income                                            Rs.  1200000.00|
-----------------------------------------------------------------------------------------
|Calculation of Income Tax                                                               |
|  Income <= 300000.00 : NIL                                                              NA|
|  300000.00 <= Income < 700000.00   : 5% of (Income - 300000)                            NA|
|  700000.00 <= Income < 1000000.00  : 20000 + 10% of (Income - 700000)                   NA|
|  1000000.00 <= Income < 1200000.00 : 50000 + 15% of (Income - 1000000)                  NA|
|  1200000.00 <= Income < 1500000.00 : 80000 + 20% of (Income - 1200000)   Rs.    80000.00|
|  Income > 1500000.00               : 140000 + 30% of (Income - 1500000)                 NA|
-----------------------------------------------------------------------------------------
|Rebate Total Income less than Rs. 7 Lakhs.                              -Rs.        0.00|
-----------------------------------------------------------------------------------------
|Total Tax                                                                Rs.    80000.00|
|Education & Health Cess @ 4% on Income Tax                               Rs.     3200.00|
|Add: Surcharge @ 10% (if income => 50 Lakhs)                                        NA|
|Add: Surcharge @ 15% (if income is more than => 1 Crore)                            NA|
-----------------------------------------------------------------------------------------
|Total tax payable                                                        Rs.    83200.00|
-----------------------------------------------------------------------------------------
```

## Numerical verification

```
Gross Salary              = Rs 1275000.00
Standard Deduction        = Rs   75000.00
Taxable Income            = Rs 1200000.00   (falls in slab: 12L <= I < 15L)

Slab tax = 80000 + 20% * (1200000 - 1200000)
         = 80000 + 0
         = Rs 80000.00

Rebate u/s 87A            = Rs 0.00          (income > 7L)
Tax after rebate          = Rs 80000.00
Surcharge                 = Rs 0.00          (income <= 50L)
Health & Edu Cess @ 4%    = 0.04 * 80000
                          = Rs 3200.00

TOTAL TAX PAYABLE         = 80000 + 3200 = Rs 83200.00   ✓
```

> ***Note:*** *The program was also tested with gross = Rs 60,75,000 (taxable 60L) → total tax Rs 17,04,560 (the 10% surcharge branch triggers), and with gross = Rs 6,75,000 (taxable 6L) → total tax Rs 0 (the 87A rebate branch triggers). All three cases produced the correct results.*

— End of Assignment —